

第17回 コマンドを選んで戦う

今回のゴール

- 配列にコマンドの一覧を入れ、`for` 文と `length` で表示できる。
- `switch` 文で、選んだコマンドごとに処理を分けられる。
- `do-while` 文で、正しい番号が入力されるまで聞き直せる。
- 新しいコマンド「ぼうぎょ」を自分で追加できる。

前回とのつながり

第16回のバトルは、勇者が自動で攻撃するだけだった。今回は、プレイヤーが毎ターン、コマンドを選べるようにする。

実習時のフォルダ構成

```
ワークスペース
├─ java17_sample
│   └─ src
│       └─ Main.java
└─ java17_exercise
    └─ src
        └─ Main.java
```

- 配布ファイル：`プログラム/第17回/sample/Main.java`、`プログラム/第17回/exercise/Main.java`
- 解答例：`プログラム/第17回/answer/Main.java`
- プロジェクトとクラスの作り方は、第16回と同じ。

今日の時間の目安

時間	内容
35 分	講義:Scanner・配列・do-while・switch、サンプルの予想と実行、エラー体験
40 分	演習1・演習2
15 分	確認問題・振り返り

- 新しく出てくる項目が 4 つ(Scanner・配列・switch・do-while)ある。Scanner は「3 行の形を使うだけ」、do-while は「読んで意味が分かればよい」として、説明を短くしてよい。
- 時間が足りない場合は、演習2-1(`for` 文で一覧を表示)と、演習2-2の「ぼうぎょ」を配列と `case 4` に追加して表示するところまでを必須とし、ダメージを半分にする処理は発展(任意)として扱ってよい。

講義内容

1. キーボードから入力する:Scanner

プレイヤーの入力を受け取るために、`Scanner` という道具を使う。この授業では、次の 3 行の形だけ覚えればよい。

```
import java.util.Scanner;           // ファイルの一番上を書く

Scanner scanner = new Scanner(System.in); // 入力の道具を用意する
int command = scanner.nextInt();       // 整数を 1 つ読み取る
```

- `Scanner` は Java に最初から用意されている道具だが、`java.util` という場所にあるので、使う前に `import` の行が必要になる(`import` は第24回でくわしく学ぶ)。
- Pleiades では、コンソールをクリックしてから数字を入力し、`Enter` を押す。

2. 配列:同じ種類の値をまとめる

コマンドの名前を、1 つの変数にまとめて入れる。

```
String[] commands = { "攻撃", "薬草を使う", "にげる" };
```

書き方	意味	値
<code>commands[0]</code>	0 番目の要素	"攻撃"
<code>commands[2]</code>	2 番目の要素	"にげる"
<code>commands.length</code>	要素の数	3

- 番号(添字)は **0 から始まる**。3 個の配列の最後は `commands[2]`。
- `commands[3]` のように、ない番号を使うと、実行時にエラーになる。

`for` 文と組み合わせると、全部の要素を順に表示できる。

```
for (int i = 0; i < commands.length; i++) {
    System.out.println((i + 1) + ": " + commands[i]);
}
```

- `i` は 0、1、2 と変わる。画面には 1 から表示したいので `i + 1` にしている。
- `i < commands.length` と書いておくと、コマンドを増やしても `for` 文を直さなくてよい。

3. do-while:最低 1 回は実行する

```
int command;
do {
    System.out.print("コマンドを選ぶ > ");
    command = scanner.nextInt();
} while (command < 1 || command > commands.length);
```

- `do { } while (条件);` は、先に 1 回実行してから 条件を調べる。入力は最低 1 回は必要なので、`do-while` が向いている。
- `||` は「または」。1 より小さい、または コマンドの数より大きい番号なら、もう一度聞く。
- 最後の `;` を忘れやすい。

4. switch: 番号ごとに処理を分ける

```
switch (command) {  
    case 1:  
        // 攻撃の処理  
        break;  
    case 2:  
        // 薬草の処理  
        break;  
    case 3:  
        // にげる処理  
        break;  
}
```

- `command` の値と同じ `case` から実行が始まる。
- `break` がないと、次の `case` の処理まで続けて実行されてしまう。
- `switch` の中の `break` は、`switch` を抜けるだけで、`while` ループは抜けない。そこで「にげた」ことを `boolean` 型の変数 `escaped` に記録し、`switch` の後で `while` を抜けている。

5. サンプルを読む

プログラム/第17回/sample/Main.java

```
import java.util.Scanner;  
  
public class Main {  
    public static void main(String[] args) {  
        // キーボードから入力するための道具  
        Scanner scanner = new Scanner(System.in);  
  
        // コマンドの一覧 (配列)  
        String[] commands = { "攻撃", "薬草を使う", "にげる" };  
  
        // 勇者のステータス  
        String heroName = "勇者";  
        int heroHp = 30;  
        int heroMaxHp = 30;  
        int heroAttack = 8;  
        int herbCount = 2;  
  
        // 敵のステータス  
        String enemyName = "ゴブリン";  
        int enemyHp = 40;  
        int enemyAttack = 7;  
  
        int turn = 1;
```

```

boolean escaped = false;

System.out.println(enemyName + "があらわれた！");

while (heroHp > 0 && enemyHp > 0) {
    System.out.println();
    System.out.println("--- ターン" + turn + " ---");
    System.out.println(heroName + " HP:" + heroHp + "/" + heroMaxHp + " 薬草:" + herbCount +
        "個");
    System.out.println(enemyName + " HP:" + enemyHp);

    // コマンドの一覧を表示する
    for (int i = 0; i < commands.length; i++) {
        System.out.println((i + 1) + ": " + commands[i]);
    }

    // 1~コマンドの数 の番号が入力されるまで、くり返し聞く
    int command;
    do {
        System.out.print("コマンドを選ぶ > ");
        command = scanner.nextInt();
    } while (command < 1 || command > commands.length);

    // 選んだコマンドごとの処理
    switch (command) {
        case 1:
            int damage = heroAttack;
            if (turn % 3 == 0) {
                damage = heroAttack * 2;
                System.out.println("会心の一撃！");
            }
            enemyHp -= damage;
            System.out.println(heroName + "の攻撃！ " + enemyName + "に" + damage + "のダメージ");
            break;
        case 2:
            if (herbCount > 0) {
                heroHp += 15;
                if (heroHp > heroMaxHp) {
                    heroHp = heroMaxHp;
                }
                herbCount--;
                System.out.println(heroName + "は薬草を使った！ HPが" + heroHp + "になった");
            } else {
                System.out.println("薬草がない！");
            }
            break;
        case 3:
            System.out.println(heroName + "はにげだした！");
            escaped = true;
            break;
    }

    // にげた、または敵をたおしたら、ループを抜ける
    if (escaped || enemyHp <= 0) {
        break;
    }
}

```

```

    }

    // 敵の攻撃
    heroHp -= enemyAttack;
    System.out.println(enemyName + "の攻撃！ " + heroName + "に" + enemyAttack + "のダメージ");
    turn++;
}

// 結果の表示
System.out.println();
if (escaped) {
    System.out.println("うまくにげきれた。");
} else if (enemyHp <= 0) {
    System.out.println(enemyName + "をたおした！");
} else {
    System.out.println(heroName + "はたおれてしまった...");
}
}
}

```

6. 実行する前に予想する

次の順に入力したとき、どうなるか予想する：1、1、1、5、2、1

- 5を入力したとき、何が起こるか。
- 薬草を使った後の HP はいくつか。
- 何ターン目に勝つか。

7. 実行結果

ゴブリンがあらわれた！

--- ターン1 ---

勇者 HP:30/30 薬草:2個

ゴブリン HP:40

1: 攻撃

2: 薬草を使う

3: にげる

コマンドを選ぶ > 1

勇者の攻撃！ ゴブリンに8のダメージ

ゴブリンの攻撃！ 勇者に7のダメージ

--- ターン2 ---

勇者 HP:23/30 薬草:2個

ゴブリン HP:32

1: 攻撃

2: 薬草を使う

3: にげる

コマンドを選ぶ > 1

勇者の攻撃！ ゴブリンに8のダメージ

ゴブリンの攻撃！ 勇者に7のダメージ

--- ターン3 ---

```
勇者 HP:16/30 薬草:2個
ゴブリン HP:24
1: 攻撃
2: 薬草を使う
3: にげる
コマンドを選ぶ > 1
会心の一撃！
勇者の攻撃！ ゴブリンに16のダメージ
ゴブリンの攻撃！ 勇者に7のダメージ

--- ターン4 ---
勇者 HP:9/30 薬草:2個
ゴブリン HP:8
1: 攻撃
2: 薬草を使う
3: にげる
コマンドを選ぶ > 5
コマンドを選ぶ > 2
勇者は薬草を使った！ HPが24になった
ゴブリンの攻撃！ 勇者に7のダメージ

--- ターン5 ---
勇者 HP:17/30 薬草:1個
ゴブリン HP:8
1: 攻撃
2: 薬草を使う
3: にげる
コマンドを選ぶ > 1
勇者の攻撃！ ゴブリンに8のダメージ

ゴブリンをたおした！
```

- `5` は範囲外なので、`do-while` がもう一度「コマンドを選ぶ」と聞いている。
- 別の順番(例:最初に `3`)でも実行して、結果を確かめる。

8. エラーを体験する

サンプルを実行し、コマンドの入力で `a` のような **数字以外** を入力する。

- コンソールに赤い文字で `Exception in thread "main" java.util.InputMismatchException` と表示され、プログラムが止まる。
- 赤い文字の中の `Main.java:` の後の数字が、エラーが起きた行番号。その行を開いて、何をしている行か確認する(`nextInt()` で整数を読もうとした行)。
- このように、**実行してから起きるエラー**を「実行時エラー(例外)」という。対処の方法は第26回で学ぶ。今は「数字を入力する」と約束して進める。

演習1 サンプルを実行する

`java17_sample` を作ってサンプルを実行し、予想と結果を比べる。わざと `0` や `9` を入力して、聞き直されることも確認する。

演習2 コマンドを増やす

`java17_exercise` を作り、`プログラム/第17回/exercise/Main.java` を貼り付ける。`// TODO` の場所を変更する。

演習2-1 コマンドの一覧を表示する

- `for` 文を使って、コマンドの一覧を `1: 攻撃` の形で表示する。
- `TODO` のまま実行すると、一覧が表示されないことを先に確認しておく。

演習2-2 「ぼうぎょ」を追加する

- 配列 `commands` の最後に `"ぼうぎょ"` を追加する。
- ぼうぎょしたかどうかを表す `boolean` 型の変数(例: `defending`)を、ターンごとに `false` で用意する。
- `case 4` を追加し、`勇者は身を守っている` と表示して、変数を `true` にする。
- 敵の攻撃のとき、ぼうぎょしたターンだけダメージを半分(`enemyAttack / 2`)にする。

ヒント:

- 配列に要素を追加すると、`commands.length` が 4 になる。`for` 文と `do-while` の条件は変えなくてよい。
- 敵のダメージ用の変数を用意し、`if (defending)` のときだけ値を変える。

完成条件(`1`、`4` と入力したときの 2 ターン目):

```
--- ターン2 ---
勇者 HP:23/30 薬草:2個
ゴブリン HP:32
1: 攻撃
2: 薬草を使う
3: にげる
4: ぼうぎょ
コマンドを選ぶ > 4
勇者は身を守っている
ゴブリンの攻撃! 勇者に3のダメージ
```

- `7 / 2` は `3` (`int` どうしの割り算なので、小数点以下は切り捨て)。

発展(任意)

- 「ぼうぎょ」したターンの次のターンだけ、攻撃力が 2 倍になるようにする。
- 薬草の回復量を、変数 `herbPower` にまとめる。

確認問題

問1 出力は何か。

```
int[] hps = { 30, 45, 20 };
System.out.println(hps.length + " " + hps[1]);
```

問2 出力は何か(すべて答える)。

```
int command = 2;
switch (command) {
    case 1:
        System.out.println("攻撃");
    case 2:
        System.out.println("防御");
    case 3:
        System.out.println("逃走");
}
```

問3 出力は何か。

```
int count = 5;
do {
    count++;
} while (count < 3);
System.out.println(count);
```

■ 答え

- 問1: 3 45 (要素は 3 個。添字 1 は 2 番目の要素)
- 問2: 防御 と 逃走 (break がないので、case 2 から後ろが全部実行される)
- 問3: 6 (do-while は条件より先に 1 回実行される)

振り返り

- 配列の添字が 0 から始まることを説明できるか。
- switch の break を忘れると何が起こるか説明できるか。
- while と do-while の違いを説明できるか。

次回へのつながり

main メソッドがかなり長くなってきた。HP の表示や回復の処理を、名前の付いたまとまり(メソッド)に分けて、読みやすくする。